APEX INSTITUTE OF TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Experiment : 6

SUBJECT: Competitive Coding-1

SUBJECT CODE: 24CSP-291

SUBMITTED TO: Mr. Sourav Jaiswal

NAME: Lucky Dewangan

UID: 24BDA70061

SECTION: 24BDS-1(B)

Aim:

1. Merge Sorted Array

▶ Two sorted arrays: nums1 (size m +n) and nums2 (size n)

▶ nums1 has m valid elements and n empty slots

▶ Merge nums2 into nums1 maintaining sorted order

▶ Modify nums1 in-place

2. Sort Colors (Dutch National Flag)

▶ Array with elements: 0 (red), 1 (white), 2 (blue)

▶ Sort in-place: all 0s, then 1s, then 2s

▶ Cannot use built-in sort

▶ Ideally: single pass (O(n))

Objectives:

- To understand and apply **in-place array manipulation techniques**
- To practice **two-pointer strategies** for efficient data movement
- To design **optimal O(n) solutions** without using extra space
- To avoid built-in sorting and implement **manual sorting logic**
- To strengthen problem-solving skills for **array-based interview questions**

Pseudocode:

1. Merge Sorted Array

```
FUNCTION merge(nums1, m, nums2, n):

    i = m - 1          // last valid element in
nums1     j = n - 1          // last element in
nums2     k = m + n - 1      // last position in
nums1
```

```
    WHILE i >= 0 AND j >= 0:
IF nums1[i] > nums2[j]:
nums1[k] = nums1[i]
i = i - 1         ELSE:
            nums1[k] =
nums2[j]             j = j - 1
k = k - 1

    // If nums2 still has
elements     WHILE j >= 0:
nums1[k] = nums2[j]         j = j
- 1         k = k - 1
```

2.Sort Colors (Dutch National Flag)

```
FUNCTION sortColors(nums):
     low = 0     mid = 0
high = length(nums) - 1

    WHILE mid <= high:

        IF nums[mid] == 0:
SWAP nums[low] and nums[mid]
low = low + 1             mid = mid + 1

        ELSE IF nums[mid] == 1:
mid = mid + 1

        ELSE:   // nums[mid] == 2
SWAP nums[mid] and nums[high]
high = high - 1
```

Code:

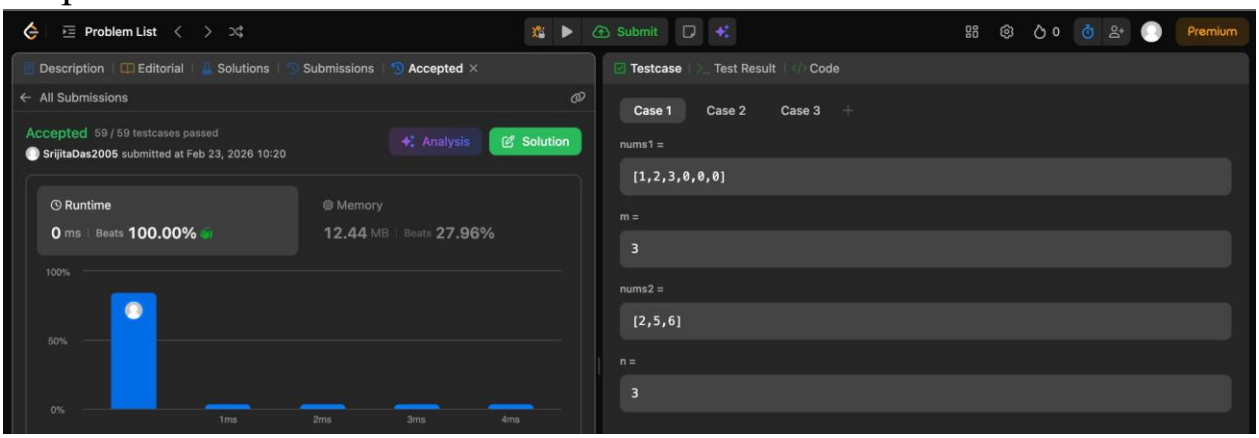
1. Merge Sorted Array

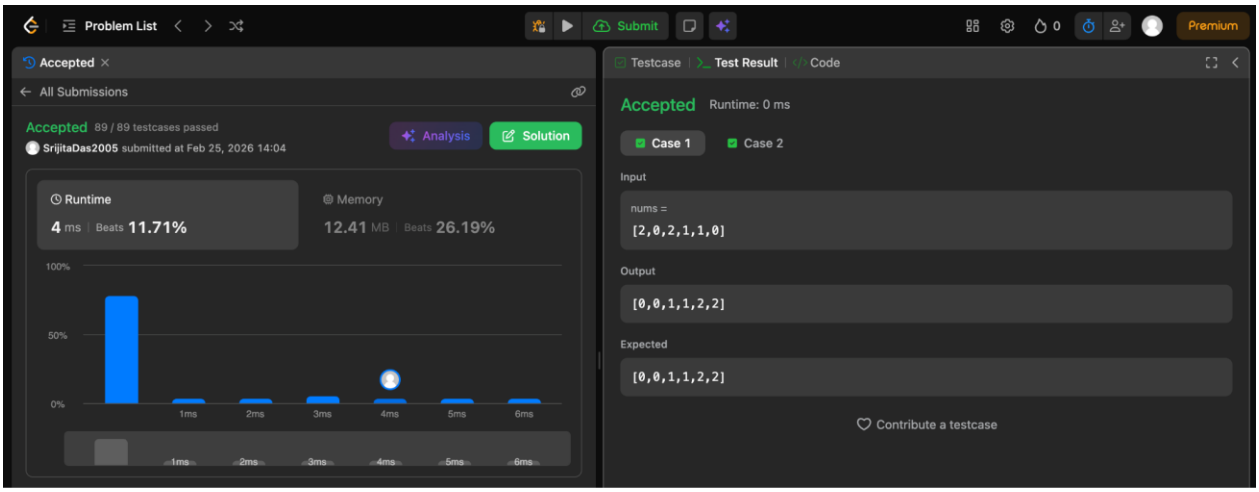
```
class Solution(object):     def
merge(self, nums1, m, nums2, n):
        i = m - 1      # last valid element in
nums1         j = n - 1      # last element in
nums2         k = m + n - 1  # last position in
nums1
         while i >= 0 and j >= 0:
if nums1[i] > nums2[j]:
nums1[k] = nums1[i]
i -= 1             else:
nums1[k] = nums2[j]
j -= 1             k -= 1

        # If nums2 still has
elements         while j >= 0:
nums1[k] = nums2[j]             j -=
1             k -= 1
```

2.Sort Colors (Dutch National Flag)

```
class Solution(object):
def sortColors(self, nums):
        low = 0         mid
= 0         high =
len(nums) - 1         while
mid <= high:             if
nums[mid] == 0:
                nums[low], nums[mid] = nums[mid],
nums[low]                 low += 1                 mid +=
1
             elif nums[mid] ==
1:
                mid += 1

            else:  # nums[mid] == 2
                nums[mid], nums[high] = nums[high], nums[mid]
high -= 1
```

Output:





Learning Outcome:

- Identify and extract **valid substrings based on conditions**
- Use **recursion and divide & conquer** for optimization
- Understand **binary representation of integers**
- Perform **bitwise operations** like shifting and masking
- Reverse bits and count set bits efficiently
- Optimize brute-force solutions into **efficient algorithms**

- Apply concepts in **coding interviews, competitive programming, and system design basics**